Lecture 9
Solutions

Homework 9: Issues, CI, and a release

Solutions to Homework 9

Part A.

```
$ bash codeHints/L09/setup_hw9.sh
$ cd practice
$ git clone hw9_server.git hw9
$ cd hw9
$ bash test.sh
FAIL: got 'Helo, world'
$ sed -i.bak 's/Helo/Hello/' greet.sh && rm greet.sh.bak    # or edit by hand
$ bash test.sh
PASS
$ git commit -am "Fix greeting typo" -m "Closes #1"
$ mkdir -p .github/workflows .github/ISSUE_TEMPLATE
```

Create `.github/workflows/ci.yml` (spaces only):

```
name: CI
on:
  push:
    branches: [main]
  pull_request:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run the tests
        run: bash test.sh
```

Create `.github/ISSUE_TEMPLATE/bug_report.md` (the front matter with `name:` and `about:` and the two headings of the lecture) and `.github/pull_request_template.md` (the heading and the checklist of the lecture). Then:

```
$ git add .github
$ git commit -m "Add CI workflow and templates"
$ git tag v0.1.0 $(git rev-list --max-parents=0 HEAD)
$ git tag -a v1.0.0 -m "First release"
$ git push origin main v0.1.0 v1.0.0
$ printf 'next-bugfix=1.0.1\nissue-closes-when=merged\n' > ../hw9_answers.txt
```

(Committing the workflow and the templates in two commits, as the task text says, works equally well.)

9.3–9.4. The checker runs your project's `test.sh` on a copy of the server's `main`, which is exactly what the workflow does on GitHub. `git commit -m ... -m ...` makes the second `-m` a separate paragraph of the message; GitHub looks for the keyword and the issue number anywhere in it.

9.5. Without `uses: actions/checkout` the virtual machine would have no files, so `bash test.sh` would fail. The workflow file must be in `.github/workflows` (with an *s*).

9.7. The lightweight tag is a bare name for the first commit; the annotated tag has its own object with your message, so `git show v1.0.0` shows a *Tagger*. Tags are only pushed when you name them (or use `--tags`).

9.8. Fixing a bug in `1.0.0` gives the patch version `1.0.1`. `Closes #12` closes the issue when the change is *merged* into the default branch, not when the pull request is opened.

Part B.

```
(GitHub) New repository: ci-practice, Public, no README
$ git remote add github git@github.com:YOURNAME/ci-practice.git
$ git push github main v0.1.0 v1.0.0
(GitHub) Actions tab: the "CI" workflow ran: green tick
$ printf 'repo=YOURNAME/ci-practice\n' >> ../hw9_answers.txt
(GitHub) Issues > New issue "Add a note to the README"  -> number 1
$ git switch -c readme-note
$ echo "Notes coming soon." >> README.md
$ git commit -am "Add a note to the README"
$ git push github readme-note
(GitHub) Open the pull request, description "Closes #1"; wait for the green check;
    Merge
(GitHub) Issue #1 is now closed
$ printf 'issue=1\n' >> ../hw9_answers.txt
(GitHub) Releases > Draft a new release > tag v1.0.0 > Generate release notes > Publish
```

Replace `YOURNAME` with your GitHub user name. The checker needs the answers file to contain `repo=`, `issue=`, and a remote of `practice/hw9` that points at that repository. It then asks GitHub's public interface whether the issue is closed, whether a release exists for `v1.0.0`, and whether the latest Actions run on `main` succeeded. If the last check is red, open the run's log in the Actions tab.

Common mistakes.

- A *tab* character in the workflow (the checker reports it) or a folder called `.github/workflow` instead of `workflows`.
- Creating Issue #1 *before* the first push: the old commit message `Closes #1` then closes it at once, and the pull request closes nothing. The check passes anyway, but you missed the exercise. Push first, then create the issue.
- Forgetting to push the tags: `git push origin main` sends no tags.
- Creating the GitHub repository with a README: the first push is rejected because the histories differ. Create it empty.
- A private repository: GitHub's public interface cannot see it, so the checker reports that it cannot find it.
- Writing `issue-closes-when=opened`: the issue closes when the change is *merged*.